

# MERN STACK LAB ACTIVITY

## Full-Stack Cloud Architectures

### Lab Activity: Student Notes CRUD Micro-App

*End-to-End React State Management, REST Endpoints, and MongoDB Persistence*

## 1. Objective & Architectural Scope

In this lab session, you will design and implement a decoupled, end-to-end full-stack notes management application using the MERN stack. The objective is to solidify practical competencies across client-server separation, RESTful HTTP contract design, asynchronous database I/O via an ODM, and reactive UI state reconciliation without manual page refreshes.

**Mongoose Schema Specification:** Persisted documents in MongoDB must strictly adhere to the following schema definition:

```
{
  title:      String,    // Heading / Subject of the note (required)
  content:    String,    // Detailed body text (required)
  createdAt:  Date       // Default: Date.now
}
```

## 2. Environment & Project Scaffolding

Initialize a clean, two-tier monorepo structure from your terminal:

```
# 1. Initialize Server Directory and Express Workspace
mkdir notes-app && cd notes-app
mkdir server && cd server
npm init -y
npm install express mongoose cors

# 2. Scaffold React Client using Vite (In a new terminal inside notes-app/)
npx create-vite@latest client -- --template react
cd client
npm install
npm install axios
```

### Port Allocation & Explicit CORS Configuration:

Your Express server should run on port 5000 (or 8000), while the Vite development server serves React on port 5173. Because client requests cross domains/ports, your Express application must explicitly mount the `cors` middleware before your route handlers:

```
const cors = require("cors");
app.use(cors()); // Permits cross-origin requests from http://localhost:5173
```

Failing to mount this middleware will cause the browser to block Axios requests with a Cross-Origin Resource Sharing (CORS) error.

### Database Connection Configuration:

Configure your server to connect to a local MongoDB daemon at `mongodb://localhost:27017/notes_db`.

Implement resilient error handling by chaining `.catch()` to your `mongoose.connect()` call so connection issues produce clear logs in the terminal.

### 3. Implementation Tasks & Technical Specifications

#### Part A: Backend API Layer (Express + MongoDB)

1. **Mongoose Data Model:** Create `models/Note.js` configuring schema properties: `title` (`String`, required), `content` (`String`, required), and `createdAt` (`Date`, default `Date.now`).
2. **RESTful Routing Endpoints:**
  - `POST /api/notes` — Parse the request payload, persist the new document, and return HTTP status 201 `Created` along with the serialized note entity.
  - `GET /api/notes` — Query and return all notes, ordered chronologically descending (`{ createdAt: -1 }`).
  - `DELETE /api/notes/:id` — Locate the document by its MongoDB `_id` parameter, execute deletion, and return HTTP 200 `OK` or HTTP 404 `Not Found` if absent.
3. **CORS and Middleware Integration:** Mount `cors()` middleware and parse incoming requests using `express.json()`.
4. **Endpoint Smoke-Testing:** Verify all three REST endpoints using Postman, Thunder Client, or `curl` prior to writing frontend components.

#### Part B: Client UI Layer (React Hooks + Axios)

1. **Controlled Submission Form:** Implement a note creation form bound to local component state (`title`, `content`). On submit, dispatch an asynchronous `POST` request via `Axios`, clear inputs, and update the displayed list.
2. **Lifecycle Data Ingestion:** Utilize React's `useEffect` hook paired with `useState` to fetch all notes on initial component mounting. Render title, content, and a formatted localized date.
3. **Interactive Deletion:** Attach a “Delete” button to each note card that invokes `DELETE /api/notes/:id` and synchronizes local state so the deleted note is removed immediately without requiring a full browser refresh.

#### Part C: Defensive States

1. **Loading and Empty States:** Display an informative loading indicator while HTTP promises are unresolved, and render a graceful fallback message (*“No notes yet — add one above!”*) when the notes list is empty.

### 4. Repository Structure Specification

You must structure your submission as a cleanly separated two-tier monorepo:

```
notes-app/
|-- .gitignore
|-- README.md           # Setup commands, candidate details & run notes
|-- screenshots/        # Visual proof of functionality (CRUD in action)
|   |-- ui-preview.png
```

```
|  |-- delete-action.png
|-- server/                                # Node.js + Express Backend Layer
|  |-- config/
|  |  |-- db.js                            # Mongoose DB connection logic
|  |-- models/
|  |  |-- Note.js                          # Mongoose Schema & model definition
|  |-- routes/
|  |  |-- noteRoutes.js                    # REST API route handlers
|  |-- package.json
|  \-- server.js                           # Express app entry point & middleware
\-- client/                                # Vite + React Client Layer
    |-- index.html
    |-- vite.config.js
    |-- package.json
    |-- src/
        |-- App.jsx                        # State orchestration, Form & Note List
        |-- main.jsx                       # React DOM root mounting
        |-- index.css                      # (Optional) Basic styling
```

## 5. Submission Protocol & Asynchronous Evaluation

For evaluation, strict adherence to packaging standards is mandatory:

- Exclusion of Dependencies:** Compress the notes-app root directory into a zip file named StudentID\_MERN\_Lab.zip. **Do NOT include node\_modules/ or dist/ folders.** The evaluator will regenerate dependencies using `npm install` based on your `package.json`.
- Package Scripts Requirement:** Ensure both `server/package.json` and `client/package.json` declare runnable scripts:
  - Server: `"start": "node server.js"`
  - Client: `"dev": "vite"`
- Database Portability:** Ensure database connectivity defaults directly to `mongodb://localhost:27017/notes` without requiring customized secret keys or non-standard ports.
- Visual Verification Proof:** Place two screenshots into a root `screenshots/` folder:
  - `ui-preview.png`: Browser view showing at least two notes successfully rendered.
  - `delete-action.png`: Browser view after deleting an item, with the DevTools Network tab showing the successful 200 OK from `DELETE /api/notes/:id`.
- README Documentation:** Include your student credentials, GitHub repository link, and step-by-step setup commands in `README.md`.